LevelUp Cloud Trainings
Sateesh Pabbathi | Microsoft Certified Trainer (MCT)

PySpark DataFrame

Module 04 — Filtering, Sorting & Lazy Evaluation

Expert Class Notes

filter() / where() | Conditions & Operators | isin() | startsWith / endsWith | isNull / isNotNull | orderBy() / sort() | Multi-column Sort | Chaining | Spark SQL | limit() | Lazy Evaluation

Filtering	Sorting	Chaining	Lazy Evaluation
7 condition patterns	Single & multi-column	Filter + Sort + Limit	Pandas vs Spark

www.levelupedu.net

ABOUT THE AUTHOR

Microsoft Certified Trainer | Data Engineering Educator | 22+ Years Industry Experience

Sateesh Pabbathi
MCT
www.levelupedu.net

Sateesh Pabbathi

Microsoft Certified Trainer (MCT)

With 22+ years in data and cloud technologies, Sateesh is the founder of LevelUp Cloud Trainings — India's premier data engineering education platform with 5,000+ trained students.

DataFrame 04 covers filtering, sorting, chaining, Spark SQL, and lazy evaluation — the operations that form the core of every daily data engineering task. These are tested in 95% of PySpark interviews.

Core Expertise

- Microsoft Fabric (DP-600) | Apache Spark & PySpark
- Databricks | Azure Data Factory | GCP & AWS
- SQL Server | Python | Big Data & Hadoop Ecosystem

"I don't just teach technology — I build careers."

5,000+	22+	10+
Students Trained	Years Experience	Technologies Taught

Module 1: The Class Datasets — Setup and Context

Before diving into filtering and sorting operations, let us understand the two datasets used throughout this class session. Understanding your data first is a professional habit — never write transformations without knowing your schema.

> **Sateesh Says 💡**
>
> *In every real project — whether at Wipro, Infosys or Amazon — you receive two types of data files: CSV (structured, tabular) and JSON (semi-structured, nested).*
>
> *The person_data.csv and employee_data.csv/json from class represent the two most common real-world data ingestion scenarios.*
>
> *Notice that Sateesh reads the SAME employee data from both CSV and JSON — this teaches you that Spark's API is IDENTICAL regardless of the file format!*

1.1 Reading CSV and JSON — The Class Setup

Reading the class datasets

```
from pyspark.sql import SparkSession
from pyspark.sql.functions import col

spark = SparkSession.builder.appName('DF04').getOrCreate()

# Dataset 1: Person data (CSV)
df = spark.read.csv('/content/person_data.csv', header=True)
df.show()

# Dataset 2: Employee data from CSV
df1 = spark.read.csv('/content/employee_data.csv', header=True)
df1.show()

# SAME data from JSON — Spark API is identical!
df2 = spark.read.json('/content/employee_data.json')
df2.show()

# Production-grade reading (always specify schema + options)
from pyspark.sql.types import *
employee_schema = StructType([
    StructField('name',       StringType(),  True),
    StructField('age',        IntegerType(), True),
    StructField('salary',     DoubleType(),  True),
    StructField('department', StringType(),  True),
])
df1 = spark.read.schema(employee_schema)  \
            .option('header', 'true')       \
            .csv('/content/employee_data.csv')
```

1.2 The Assignment Dataset — Created Inline

The class also creates a dataset directly in Python for the assignment exercise. This pattern is used in unit tests and quick exploratory analysis.

Assignment dataset — created from Python dict (from class)

```
assignment_data = {
    'name':       ['Alice','Bob','Charlie','David','Eva',
                   'Frank','Grace','Helen','Ivy','Jack'],
    'age':        [34, 28, 45, 23, 31, 29, 41, 35, 26, 38],
    'salary':     [55000, 42000, 70000, 31000, 48000,
                   46000, 68000, 59000, 39000, 61000],
    'department': ['Sales','IT','HR','IT','Finance',
                   'Finance','HR','Sales','IT','HR']
}

# Convert dict to list of tuples
data = list(zip(assignment_data['name'],
                assignment_data['age'],
                assignment_data['salary'],
                assignment_data['department']))
columns = ['name', 'age', 'salary', 'department']

df = spark.createDataFrame(data, schema=columns)
df.show()
```

Assignment DataFrame

```
+-------+---+------+----------+
|   name|age|salary|department|
+-------+---+------+----------+
|  Alice| 34| 55000|     Sales|
|    Bob| 28| 42000|        IT|
|Charlie| 45| 70000|        HR|
|  David| 23| 31000|        IT|
|    Eva| 31| 48000|   Finance|
|  Frank| 29| 46000|   Finance|
|  Grace| 41| 68000|        HR|
|  Helen| 35| 59000|     Sales|
|    Ivy| 26| 39000|        IT|
|   Jack| 38| 61000|        HR|
+-------+---+------+----------+
```

Module 2: filter() and where() — Selecting Rows

filter() and where() are identical — they are aliases of each other. Both create a new DataFrame with only the rows that satisfy the condition. This is a NARROW transformation — no shuffle, runs in parallel on every partition.

Sateesh Says 💡

Think of filter() like a quality control inspector at an Infosys assembly line.

Every product (row) passes by the inspector (condition). If it passes inspection (condition = True) it moves forward. If not (False), it gets discarded.

The key insight: filter() is CHEAP because it runs independently on each partition. No data movement needed.

Always filter() FIRST in your pipeline before any expensive operations like groupBy or join. Reduce data volume early!

2.1 Basic Filtering — From Class

Pattern 1: Simple comparison (from class)

```
# From class: person_data — filter age > 40
df.filter(df['Age'] > 40).show()

# Equivalent ways to write the same filter:
df.filter(df['Age'] > 40).show()             # df['col'] style
df.filter(col('Age') > 40).show()            # col() style
df.filter('Age > 40').show()                 # SQL string style
df.where(df['Age'] > 40).show()              # where() = filter()
df.where('Age > 40').show()                  # where + SQL string

# All produce IDENTICAL results and IDENTICAL execution plans!
```

2.2 Multiple Conditions — AND, OR, NOT

Sateesh Says 💡

The MOST common beginner bug in PySpark: using Python's 'and' and 'or' instead of '&' and '|'.

Python's 'and' evaluates the truthiness of the whole Column object (always True) — not the element-wise condition.

You MUST use '&' for AND, '|' for OR, and '~' for NOT. And wrap EACH condition in parentheses.

This is tested in EVERY PySpark interview. Interviewers intentionally ask this to catch beginners!

Multiple conditions — AND & OR (from class)

```
# From class: filter age < 40 AND first_name = 'Lucy'
df.filter((df['Age'] < 40) & (df['First_Name'] == 'Lucy')).show()
# ^ Parentheses around EACH condition are MANDATORY

# --- AND conditions ---
# Method 1: & operator (most common)
df1.filter((df1['age'] > 30) & (df1['salary'] > 40000)).show()

# Method 2: SQL string (very readable)
df1.filter('age > 30 AND salary > 40000').show()
df1.where('age > 30 AND salary > 40000').show()

# Method 3: Chain multiple .filter() calls (clean, Pythonic)
df1.filter(df1['age'] > 30) \
   .filter(df1['salary'] > 40000).show()
# Chaining filters = same result as &, often more readable

# --- OR conditions ---
df1.filter((df1['department'] == 'HR') | (df1['department'] == 'IT')).show()
df1.filter('department = "HR" OR department = "IT"').show()

# --- NOT condition ---
df1.filter(~(df1['department'] == 'HR')).show()   # NOT HR
df1.filter(df1['department'] != 'HR').show()      # same thing
df1.filter('department != "HR"').show()           # SQL style

# --- Complex: AND + OR combined ---
df1.filter(
    (df1['age'] > 30) &
    ((df1['department'] == 'Finance') | (df1['department'] == 'HR'))
).show()
```

Common Mistake — Watch Out!

WRONG: df.filter(df['age'] > 30 and df['salary'] > 40000) <- Python 'and', evaluates Column objects, ALWAYS True!

RIGHT: df.filter((df['age'] > 30) & (df['salary'] > 40000)) <- Element-wise AND with parentheses

WRONG: df.filter(df['city'] == 'Mumbai' | df['city'] == 'Delhi') <- Missing parentheses, operator precedence error

RIGHT: df.filter((df['city'] == 'Mumbai') | (df['city'] == 'Delhi')) <- Each condition in its own parentheses

2.3 isin() — Filter by a List of Values

isin() is the PySpark equivalent of SQL's IN operator. It filters rows where the column value matches ANY of the provided values. It is much cleaner than chaining multiple OR conditions.

isin() — From class and extended patterns

```
# FROM CLASS: filter Finance or IT departments
df1.filter(df1['department'].isin('Finance', 'IT')).show()
```

```
# Passing a Python list (useful for dynamic lists)
target_depts = ['Finance', 'IT', 'HR']
df1.filter(df1['department'].isin(target_depts)).show()
df1.filter(col('department').isin(target_depts)).show()

# NOT IN — negate with ~ (tilde)
df1.filter(~df1['department'].isin('Finance', 'IT')).show()
df1.filter(~col('department').isin(['Finance','IT'])).show()

# isin() with numeric values
df.filter(col('age').isin([25, 30, 35, 40])).show()
df.filter(col('id').isin(1, 3, 5)).show()

# SQL equivalent (also valid in PySpark):
df1.filter("department IN ('Finance', 'IT')").show()
df1.where("department NOT IN ('Finance', 'IT')").show()
```

isin Finance or IT output (from class dataset)

```
# Shows only Finance and IT employees
# Example: Eva (Finance), Frank (Finance), Bob (IT), David (IT), Ivy (IT)
```

2.4 startsWith() and endsWith() — String Pattern Filters

These methods filter rows based on whether a string column starts or ends with a specific prefix or suffix. They are faster than LIKE '%pattern%' because they can be optimized at the storage level.

startsWith() and endsWith() — from class

```
# FROM CLASS: employees whose name ends with 'y'
df1.filter(df1['name'].endswith('y')).show()
# Matches: Betty, Mary, Anthony (any name ending in 'y')

# startswith — employees whose name starts with 'A'
df1.filter(df1['name'].startswith('A')).show()
# Matches: Alice, Anthony, Ananya etc.

# Case-insensitive version
df1.filter(col('name').startswith('a')).show()  # only lowercase a
# To make case-insensitive:
from pyspark.sql.functions import lower, upper
df1.filter(lower(col('name')).startswith('a')).show()

# contains() — check if string CONTAINS a substring
df1.filter(col('email').contains('@gmail.com')).show()
df1.filter(col('name').contains('ar')).show()  # Sharma, Archana

# like() — SQL LIKE pattern matching
df1.filter(col('name').like('A%')).show()      # starts with A
df1.filter(col('name').like('%son')).show()     # ends with 'son'
df1.filter(col('name').like('%ari%')).show()    # contains 'ari'
df1.filter(col('name').like('_avi')).show()     # 4-char, ends 'avi'

# rlike() — regex pattern matching
df1.filter(col('email').rlike(r'^[a-z]+@gmail\.com$')).show()
```

2.5 isNull() and isNotNull() — Filter Missing Data

Null-based filtering is critical for data quality pipelines. These are the correct ways to check for null values — equality operators (== None, != None) do NOT work correctly in PySpark.

isNull() and isNotNull() — from class

```
# FROM CLASS: filter rows where department is NOT null
df1.filter(df1['department'].isNotNull()).show()
df1.filter(col('department').isNotNull()).show()  # same
df1.filter('department IS NOT NULL').show()       # SQL style

# Filter rows WHERE department IS null
df1.filter(df1['department'].isNull()).show()
df1.filter('department IS NULL').show()           # SQL style

# Combine null check with other conditions
df1.filter(col('salary').isNotNull() & (col('age') > 30)).show()

# Count null vs non-null
print('Total rows:',   df1.count())
print('Null dept:',    df1.filter(col('department').isNull()).count())
print('NonNull dept:', df1.filter(col('department').isNotNull()).count())

# NULL comparison trap — NEVER use == None for null check
# WRONG: df1.filter(col('department') == None).show() <- returns empty!
# RIGHT: df1.filter(col('department').isNull()).show()
```

Common Mistake — Watch Out!

Never compare with None using == in PySpark column expressions. In SQL/PySpark, NULL == NULL is FALSE (not True).

col('salary') == None returns an empty DataFrame because NULL == NULL evaluates to NULL, not True.

Always use col('salary').isNull() to find null values and col('salary').isNotNull() to find non-null values.

2.6 between() — Range Filtering

between() — filter within a numeric or date range

```
# Filter ages between 25 and 40 (inclusive on both ends)
df.filter(col('age').between(25, 40)).show()
df.filter('age BETWEEN 25 AND 40').show()   # SQL style

# Filter salary range
df.filter(col('salary').between(40000, 60000)).show()

# Filter date range
df.filter(col('join_date').between('2022-01-01', '2022-12-31')).show()

# EQUIVALENT to:
df.filter((col('age') >= 25) & (col('age') <= 40)).show()
```

2.7 Complete Filter Methods Reference

Method	Syntax	SQL Equivalent	Use Case
filter() / where()	df.filter(condition)	WHERE condition	Primary row selection
& (AND)	(cond1) & (cond2)	AND	Both conditions must be True
| (OR)	(cond1) | (cond2)	OR	Either condition True
~ (NOT)	~(condition)	NOT / !=	Negate a condition
isin()	col.isin(['A','B'])	IN ('A','B')	Match any of a list
startsWith()	col.startsWith('A')	LIKE 'A%'	Prefix match
endsWith()	col.endsWith('y')	LIKE '%y'	Suffix match
contains()	col.contains('ar')	LIKE '%ar%'	Substring match
like()	col.like('A%')	LIKE 'A%'	SQL LIKE wildcard
rlike()	col.rlike('^[a-z]+')	REGEXP	Regular expression
isNull()	col.isNull()	IS NULL	Find missing values
isNotNull()	col.isNotNull()	IS NOT NULL	Find present values
between()	col.between(a, b)	BETWEEN a AND b	Inclusive range

2.8 Interview Questions — Filtering

IQ-01 **What is the difference between filter() and where() in PySpark?**

They are IDENTICAL — where() is simply an alias for filter(). Both create a new DataFrame with only the rows satisfying the condition. Both are NARROW transformations (no shuffle). Both accept the same argument types: Column expression, string SQL expression, or Python boolean expression on Column objects. Use whichever reads more naturally — SQL-background developers often prefer where().

IQ-02 **Why can't you use Python's 'and' and 'or' in PySpark filter conditions?**

Python's 'and'/'or' evaluate the truthiness of the entire Column object, which is always True (a Column object is non-None). They don't perform element-wise comparison on each row's value. You must use '&' for AND and '|' for OR — these are overloaded operators that produce Column expressions for element-wise boolean operations. Also always wrap each condition in parentheses: (cond1) & (cond2), because & has lower precedence than == in Python.

IQ-03 **How does isin() differ from chaining multiple OR conditions?**

isin(['A','B','C']) is semantically equivalent to (col=='A') | (col=='B') | (col=='C') but is dramatically more readable and maintainable. For 10+ values, isin() from a Python list is far cleaner than chaining ORs. Under the hood, Catalyst may optimize isin() as a hash lookup rather than sequential OR evaluation, making it potentially faster on large lists. isin() also handles null values consistently — if the list contains None, rows with null values match.

IQ-04 What is the difference between startsWith() and like() in PySpark?

startsWith('A') checks if the string begins with 'A' — simple prefix match. like('A%') uses SQL LIKE syntax with wildcards: % matches any sequence of characters, _ matches exactly one character. like() is more powerful: like('%son') (endsWith), like('%ari%') (contains), like('_avi') (4-char ending in 'avi'). rlike() goes further with full regex support. For simple prefix/suffix checks, startsWith()/endsWith() are cleaner; for complex patterns use like() or rlike().

Module 3: orderBy() and sort() — Sorting Data

Sorting arranges rows in a specific order based on column values. orderBy() and sort() are identical — both sort the entire DataFrame globally. Unlike filter(), sorting is a WIDE transformation — it requires a shuffle to globally order data across all partitions.

> **Sateesh Says** 💡
>
> *Think of sorting like ranking students in a class of 1000. You can't rank them when they're sitting in 20 different rooms (partitions).*
>
> *You first bring everyone to one big auditorium (shuffle all data), sort them by marks, then send them back to their seats.*
>
> *That is why sorting is expensive — it moves all data across the network. In PySpark: sort() triggers a full shuffle.*
>
> *Rule: always filter() and aggregate() first to reduce data volume, THEN sort() the small result. Don't sort 100 million rows to find the top 10!*

3.1 Single Column Sorting

Sort by single column — from class

```
# FROM CLASS: sort by age ascending (default)
df1.orderBy('age').show()
df1.sort('age').show()   # sort() is alias of orderBy()

# FROM CLASS: sort by age DESCENDING
df1.orderBy(df1['age'].desc()).show()

# All equivalent ways to sort descending:
df1.orderBy(col('age').desc()).show()
df1.orderBy(col('age'), ascending=False).show()
df1.sort(col('age').desc()).show()
df1.orderBy('age', ascending=False).show()

# Ascending options (all equivalent):
df1.orderBy('age').show()                       # default: ascending
df1.orderBy(col('age').asc()).show()             # explicit ascending
df1.orderBy(col('age'), ascending=True).show() # explicit flag

# Sort by string column (alphabetical)
df1.orderBy('name').show()              # A -> Z
df1.orderBy(col('name').desc()).show()  # Z -> A
```

3.2 Multi-Column Sorting — From Class

Multi-column sorting sorts by the first column, and breaks ties using the second column, third column, and so on. Each column can have its own sort direction.

Multi-column sort — from class

```
# FROM CLASS: sort by department ASC, then salary DESC within dept
df1.orderBy(df1['department'].asc(), df1['salary'].desc()).show()

# Same using col():
df1.orderBy(col('department').asc(), col('salary').desc()).show()

# Using string column names with ascending list:
df1.orderBy(['department', 'salary'], ascending=[True, False]).show()

# 3-column sort
df.orderBy(
    col('department').asc(),     # primary sort: dept A-Z
    col('salary').desc(),        # secondary: high salary first
    col('name').asc()            # tiebreaker: alphabetical by name
).show()
```

Multi-column sort result (department ASC, salary DESC)

```
+-------+---+------+----------+
|   name|age|salary|department|
+-------+---+------+----------+
|    Eva| 31| 48000|   Finance|  <- Finance first, highest salary
|  Frank| 29| 46000|   Finance|
|  Grace| 41| 68000|        HR|  <- HR second, highest salary first
|Charlie| 45| 70000|        HR|  <- Wait: 70000 > 68000 (corrected below)
|   Jack| 38| 61000|        HR|
|  Alice| 34| 55000|     Sales|  <- Sales third
|  Helen| 35| 59000|     Sales|
|Charlie| 45| 70000|        IT|  -> actually IT not HR based on data
+-------+---+------+----------+
Note: Within each department, salary goes from HIGH to LOW (desc)
```

3.3 Handling Nulls in Sorting

By default, null values appear FIRST in ascending sorts and LAST in descending sorts (Spark SQL semantics). You can control this explicitly.

Null handling in sort

```
# Default: nulls appear first in ASC sort, last in DESC sort
df.orderBy(col('salary').asc()).show()   # nulls first
df.orderBy(col('salary').desc()).show()  # nulls last

# Explicit null position control:
df.orderBy(col('salary').asc_nulls_last()).show()   # nulls at END
df.orderBy(col('salary').asc_nulls_first()).show()  # nulls at START
df.orderBy(col('salary').desc_nulls_last()).show()  # nulls at END
df.orderBy(col('salary').desc_nulls_first()).show() # nulls at START

# Best practice in production:
# Always be explicit about null position — default varies by context
df.orderBy(col('amount_paid').desc_nulls_last()).show()
# -> high amounts first, null amounts at the very end of results
```

3.4 sortWithinPartitions — Performance-Optimized Sort

sortWithinPartitions() sorts data within each partition independently — NO shuffle. Use it when you need sorted output files but don't need global ordering.

sortWithinPartitions — no shuffle, faster than orderBy

```
# Global sort (WIDE — full shuffle, slow for large data):
df.orderBy('salary').write.parquet('output/')

# Partition-level sort (NARROW — no shuffle, MUCH faster):
df.sortWithinPartitions('salary').write.parquet('output/')
# Each output file will be sorted by salary, but files themselves
# are not globally ordered. Good for sorted Parquet files.

# When to use sortWithinPartitions:
# - Writing sorted Parquet files for efficient range queries
# - Preparing data for delta merge operations
# - When you need local order but not global order
```

3.5 Interview Questions — Sorting

IQ-05 **What is the difference between sort() and orderBy() in PySpark?**

They are identical — sort() is an alias for orderBy(). Both perform a global sort of the entire DataFrame, requiring a SHUFFLE (wide transformation). Both create a new DataFrame with rows in the specified order. Both accept the same arguments: column names as strings, Column objects with .asc()/.desc(), or a list of columns with ascending=[True/False]. Use whichever reads more naturally in your code style.

IQ-06 **Is orderBy a narrow or wide transformation? Why?**

orderBy is a WIDE transformation. To globally sort a DataFrame, Spark must compare every row against every other row. Since rows are distributed across multiple partitions on different nodes, all data must be shuffled to one or a few nodes for comparison. This shuffle is expensive in terms of network I/O. The exception is sortWithinPartitions() which is NARROW — it sorts within each partition independently without any shuffle.

IQ-07 **How do you sort by multiple columns with different sort directions?**

Use: df.orderBy(col('dept').asc(), col('salary').desc()). Each column gets its own direction. Alternatively: df.orderBy(['dept','salary'], ascending=[True, False]). The list of ascending flags maps positionally to the list of columns. Primary sort is applied first; secondary sort breaks ties in the primary. Always specify null handling explicitly in production: col('salary').desc_nulls_last().

Module 4: Chaining, limit() and the Assignment Solution

Chaining is the practice of calling multiple DataFrame operations one after another in a single readable expression. It is the standard coding style in production PySpark. limit() takes the first N rows from a DataFrame — useful for sampling, top-N queries, and previewing results.

> **Sateesh Says 💡**
>
> *Chaining is like an assembly line at a Flipkart fulfillment center. Package enters, gets labeled, then scanned, then sorted, then dispatched — ONE after another.*
>
> *In PySpark: df.filter().select().withColumn().orderBy().limit() — each operation feeds its output into the next.*
>
> *CRITICAL: Because of LAZY EVALUATION, none of these execute until the final show() or write() — Spark builds ONE optimized plan for the ENTIRE chain!*

4.1 Combining filter() and orderBy() — From Class

Filter + Sort chaining — from class

```
# FROM CLASS: filter age > 30 AND salary > 40000, then sort
df1.filter(
    (df1['age'] > 30) & (df1['salary'] > 40000)
).orderBy(
    df1['department'].asc(),
    df1['salary'].desc()
).show()

# More readable multi-line version:
result = df1 \
    .filter((col('age') > 30) & (col('salary') > 40000)) \
    .orderBy(col('department').asc(), col('salary').desc())
result.show()
```

4.2 limit() — Take First N Rows

limit(n) returns a new DataFrame with only the first n rows. It is a NARROW transformation but triggers partial execution. Used for top-N queries, sampling large datasets, and paginating results.

limit() — patterns and class assignment solution

```
# Basic limit
df.limit(5).show()     # first 5 rows
df.limit(10).show()    # first 10 rows
```

```
# FROM CLASS ASSIGNMENT: Top 3 highest-paid Finance employees
df.filter(df['department'] == 'Finance') \
  .orderBy(col('salary').desc()) \
  .limit(3) \
  .show()

# On the assignment dataset:
# Finance employees: Eva(48000), Frank(46000)
# After orderBy DESC + limit(3):
# Eva 48000, Frank 46000 (only 2 Finance employees in dataset)

# Alternative: orderBy + take() instead of limit().show()
top_3 = df.filter(col('department')=='Finance') \
           .orderBy(col('salary').desc())        \
           .take(3)   # returns Python list of Row objects
for row in top_3:
    print(row)

# limit() vs take():
# limit(N) returns a DataFrame (lazy, chainable)
# take(N)  returns a Python list (action, executes immediately)
```

Assignment: Top 3 Finance employees by salary

```
+-----+---+------+----------+
| name|age|salary|department|
+-----+---+------+----------+
|  Eva| 31| 48000|   Finance|
|Frank| 29| 46000|   Finance|
+-----+---+------+----------+
(Only 2 Finance employees in assignment dataset)
```

4.3 Full Chaining Pipeline — Multiple Operations

Production-grade chaining pipeline

```
# Real-world ETL pipeline — all steps chained:
result = df \
    .filter(col('department').isin(['HR', 'Finance'])) \  # Step 1
    .filter(col('salary').isNotNull()) \                  # Step 2
    .withColumn('salary_band',                            # Step 3
        F.when(col('salary') >= 60000, 'Senior') \
         .when(col('salary') >= 45000, 'Mid')    \
         .otherwise('Junior')) \
    .select('name', 'department', 'salary', 'salary_band') \ # Step 4
    .orderBy(col('department').asc(), col('salary').desc()) \ # Step 5
    .limit(10)                                            # Step 6

result.show()   # <- ACTION: entire chain executes NOW

# None of steps 1-6 execute until .show() is called!
# Catalyst Optimizer sees the FULL chain and optimizes:
# - Columns not needed are never read from disk
# - Filters pushed down to storage layer (Parquet/Delta)
# - Steps 1+2 may be combined into one pass
```

4.4 Spark SQL for Filtering and Sorting

PySpark SQL lets you write pure SQL after registering your DataFrame as a temp view. This is the easiest way for SQL-background engineers to work with Spark — identical performance to the DataFrame API.

createOrReplaceTempView + spark.sql — from class

```
# Register the DataFrame as a SQL temporary view
df1.createOrReplaceTempView('employees')

# Now query with pure SQL — identical to SQL Server / MySQL
spark.sql('SELECT * FROM employees WHERE age > 30 ORDER BY salary DESC').show()

# Complex SQL — filter, sort, limit combined
spark.sql('''
    SELECT name, department, salary
    FROM employees
    WHERE age > 30
      AND salary > 40000
      AND department IN ('Finance', 'HR')
    ORDER BY department ASC, salary DESC
    LIMIT 5
''').show()

# The Assignment in SQL:
df.createOrReplaceTempView('staff')
spark.sql('''
    SELECT name, salary, department
    FROM staff
    WHERE department = 'Finance'
    ORDER BY salary DESC
    LIMIT 3
''').show()

# Global temp view — available across SparkSessions
df.createOrReplaceGlobalTempView('global_employees')
spark.sql('SELECT * FROM global_temp.global_employees').show()
```

4.5 Interview Questions — Chaining and limit()

IQ-08 **What is method chaining in PySpark and why is it preferred?**

Method chaining calls multiple DataFrame transformations one after another: df.filter().select().orderBy().limit(). It is preferred because: (1) All transformations are lazy — Catalyst sees the FULL chain and builds ONE optimized execution plan, potentially 10x faster than saving intermediate DataFrames. (2) Code is more readable — the data flow reads top to bottom like a pipeline. (3) No intermediate DataFrames are materialized in memory unnecessarily.

IQ-09 **What is the difference between limit() and take()?**

limit(N) is a transformation that returns a new DataFrame with the first N rows — lazy, chainable, returns DataFrame type. take(N) is an action that executes the pipeline immediately and returns a Python list of Row objects on the Driver. Both return N rows.

Use limit() when you want to chain further operations or write output. Use take() when you want to work with the results in Python immediately (loop, print, unit test assertions).

IQ-10

What is the performance implication of calling orderBy() before limit() on a 100 million row DataFrame?

orderBy() on 100M rows triggers a global shuffle — all data redistributed across the cluster. This is extremely expensive (potentially 10+ minutes). Better approach: use df.orderBy().limit(N) — Spark is smart enough to NOT sort all 100M rows; it uses a partial sort/heap that only tracks top-N elements, reducing shuffle significantly. Even better: use rdd.takeOrdered(N, key=lambda x: -x.salary) which avoids full sort entirely. Always combine sort with limit for top-N queries.

Module 5: withColumnRenamed() — Renaming Columns

withColumnRenamed() is one of the core column management functions. It renames one column at a time and returns a new DataFrame with all other columns unchanged. From the class: df.withColumnRenamed('name', 'first_name').

withColumnRenamed() — from class

```
# FROM CLASS: rename 'name' column to 'first_name'
df6 = df.withColumnRenamed('name', 'first_name')
df6.show()

# General syntax:
df.withColumnRenamed('existing_col_name', 'New_col_name').show()

# Chain multiple renames:
df.withColumnRenamed('name', 'full_name') \
  .withColumnRenamed('age', 'age_years')  \
  .withColumnRenamed('salary', 'ctc').show()

# Rename multiple columns using dict + reduce (cleaner for many cols)
from functools import reduce
rename_map = {'name':'full_name', 'age':'age_years', 'salary':'ctc'}
renamed_df = reduce(lambda df_, kv: df_.withColumnRenamed(kv[0], kv[1]),
             rename_map.items(), df)
renamed_df.show()

# Check if column exists before renaming (safe pattern)
if 'name' in df.columns:
    df = df.withColumnRenamed('name', 'first_name')

# After rename — verify:
print(df6.columns)   # ['first_name', 'age', 'salary', 'department']
```

Key Behaviours of withColumnRenamed

* Returns a NEW DataFrame — the original is NOT modified (DataFrames are immutable)
* All other columns remain UNCHANGED — only the specified column is renamed
* If the column name does not exist — NO error is thrown (silently ignored!)
* If you provide the same name for old and new — column is unchanged (no-op)
* It is a NARROW transformation — no shuffle, very fast
* Internally it is a projection (select with alias) — same Catalyst optimization

5.1 Interview Questions — withColumnRenamed

IQ-11 **What happens if you call withColumnRenamed() with a column name that**

doesn't exist?

withColumnRenamed() silently ignores the rename and returns the DataFrame unchanged — no error, no warning. Example: df.withColumnRenamed('nonexistent_col', 'new_name') returns df with no changes. This is different from withColumn() which would add a new column if the name doesn't exist. Always verify columns using df.columns or df.printSchema() before renaming in production pipelines.

IQ-12 How do you rename many columns at once efficiently in PySpark?

Three approaches: 1. Chain withColumnRenamed() calls (simple but verbose for many columns). 2. Use toDF(*new_col_names) to rename ALL columns positionally. 3. Use select() with alias: df.select([col(c).alias(rename_map.get(c,c)) for c in df.columns]) — most flexible, works with a dictionary mapping. Option 3 is best for production because it is explicit, handles missing columns gracefully, and can be unit tested easily.

Module 6: Lazy Evaluation — Spark's Superpower

Lazy evaluation is one of the most important concepts in PySpark — and the biggest difference from Pandas. Understanding it deeply is what makes the difference between writing slow code and writing production-grade optimized pipelines.

> **Sateesh Says 💡**
>
> *Sateesh wrote in the class notebook: 'Lazy evaluation -> a transformation is performed only when action is called for'.*
>
> *Compare this to Pandas: 'Eager — it performs the operation immediately'.*
>
> *Here is the analogy: Imagine you order a full Biryani banquet from a hotel for a party. You give the full menu order.*
>
> *EAGER (Pandas): The hotel cooks each dish the moment you order it. Biryani ready at 10am, raita at 10:05am, dessert at 10:10am. You wait between each dish!*
>
> *LAZY (PySpark): The hotel writes down your ENTIRE order, optimises the kitchen schedule, then cooks everything at 12pm when guests arrive. More efficient, and the chef can skip dishes nobody wants!*

6.1 Transformations vs Actions — The Core Distinction

Lazy evaluation in action

```
# Step-by-step demonstration of lazy evaluation

df = spark.read.csv('employees.csv', header=True)  # lazy: no read yet!
print('df created -- nothing executed yet')

# All transformations -- LAZY -- just build the execution plan
df2 = df.filter(col('age') > 30)           # lazy -- no rows filtered
df3 = df2.select('name', 'age', 'salary')  # lazy -- no cols selected
df4 = df3.orderBy(col('salary').desc())    # lazy -- no sort done
df5 = df4.limit(10)                        # lazy -- no limit applied

print('4 transformations added -- STILL nothing executed!')

# ACTION -- THIS triggers execution of ALL 4 transformations at once!
df5.show()   # <-- Spark NOW reads csv, filters, selects, sorts, limits

# Equivalent to show(), these also trigger execution:
df5.count()                    # count = action
df5.collect()                  # collect = action
df5.write.parquet('out/')      # write = action
df5.toPandas()                 # toPandas = action
```

6.2 How Lazy Evaluation Works — The DAG

When you write transformations, Spark builds a DAG (Directed Acyclic Graph) — an execution plan. When an action is triggered, Catalyst Optimizer rewrites and optimizes this plan before actually executing anything.

The DAG — what Catalyst sees

```
# Your code (what you wrote):
df.filter(col('age') > 30) \
  .select('name', 'department', 'salary') \
  .groupBy('department').agg(F.avg('salary')) \
  .orderBy(col('avg(salary)').desc())
  .show()

# What Catalyst optimizes it to (roughly):
# 1. Read only columns: name, department, salary, age (not all 50!)
# 2. Push age > 30 filter DOWN to the file scan (read less data)
# 3. Execute filter + select in ONE pass (pipeline)
# 4. Shuffle for groupBy (one shuffle only)
# 5. Sort the tiny aggregated result

# Check the plan with explain():
df.filter(col('age')>30).groupBy('dept').agg(F.avg('salary')).explain()
# Shows: Physical Plan -> FileScan with PushedFilters, HashAggregate
```

6.3 Pandas Eager vs PySpark Lazy — Complete Comparison

Pandas (Eager)	PySpark (Lazy)
Executes IMMEDIATELY when you write it	Executes ONLY when action is called
Single machine — RAM only	Distributed across cluster
df2 = df[df['age']>30] <-- RUNS NOW	df2 = df.filter(...) <-- no execution
No optimization — executes as written	Catalyst optimizes entire chain
df.sort_values() sorts immediately	df.orderBy() adds to plan, runs later
Small data: fast. Large data: crashes	Any size: distributes across cluster
Intermediate results stored in RAM	No intermediate materialization
When to use: < 10 GB on one machine	When to use: any size, production

6.4 Actions vs Transformations — Complete Reference

Transformations (LAZY)	Actions (TRIGGER Execution)
filter() / where()	show()

select() / selectExpr()	count()
withColumn() / drop()	collect()
groupBy().agg()	take(N)
orderBy() / sort()	first()
join() / union()	write.parquet() / csv()
withColumnRenamed()	toPandas()
repartition() / coalesce()	saveAsTextFile()
distinct() / dropDuplicates()	foreach() / foreachPartition()
limit() — returns DF (lazy!)	head(N)

6.5 Interview Questions — Lazy Evaluation

IQ-13 **What is lazy evaluation in PySpark? Why does Spark use it?**

Lazy evaluation means transformations are NOT executed when written — Spark builds a logical execution plan (DAG). Only when an ACTION is called does Spark compile the DAG, optimize it via Catalyst, and execute it. Spark uses lazy evaluation because: (1) It sees the FULL pipeline before executing, enabling optimizations like predicate pushdown, column pruning, join reordering. (2) It avoids unnecessary computation — if you write 10 transformations but only need 2 columns in the output, it never processes the other 8. (3) It enables fault tolerance via lineage.

IQ-14 **What is the difference between PySpark lazy evaluation and Pandas eager evaluation?**

Pandas executes every operation immediately ('eager') — the result is computed and stored in RAM right away. PySpark is lazy — every transformation just extends the execution plan. Practical difference: in Pandas, df2 = df[df.age > 30] immediately creates a filtered DataFrame in memory. In PySpark, df2 = df.filter(col('age')>30) creates nothing — it's just a plan node. Only when df2.show() is called does any computation happen. Lazy evaluation enables Catalyst optimization, which can make PySpark code run faster than equivalent Pandas even for some mid-size datasets.

IQ-15 **Can you give a real-world example where lazy evaluation improved performance significantly?**

Scenario: A 1 TB Parquet file with 200 columns. You write: df.select('city','salary').filter(col('city')=='Mumbai').groupBy('city').agg(avg('salary')).show() . With eager evaluation: Spark reads all 200 columns (1 TB), applies filter (still 1TB in memory), then projects down to 2 columns. With lazy evaluation and Catalyst: Spark reads ONLY 2 columns from Parquet (column pruning) and applies the filter AT READ TIME (predicate pushdown). Actual data read might be 5 GB instead of 1 TB — 200x less I/O. This is the power of Catalyst + lazy evaluation combined.

IQ-16 Is limit() a transformation or an action in PySpark?

limit() is a TRANSFORMATION — it returns a new DataFrame lazily. It does NOT execute until an action is called on the result. df.limit(10) just adds a 'take first 10' node to the execution plan. df.limit(10).show() is when execution happens. Compare with take(N) which IS an action — it immediately executes and returns a Python list. Common confusion: limit() looks like it 'does something' but it's just another plan node until an action triggers it.

Module 7: Real-World Scenarios — Applying Everything

Now let us apply all the operations from this session to realistic business scenarios. These represent the types of questions asked in PySpark coding interviews at Amazon, Flipkart, TCS Digital, and Databricks.

01 SCENARIO

Employee Analytics

Objective: Find top 5 highest-paid employees above age 30, sorted by salary desc

Operations: filter + orderBy + limit

Scenario 1: Solution

```
# Using the assignment dataset
result = df \
    .filter(col('age') > 30) \            # keep employees over 30
    .orderBy(col('salary').desc()) \      # highest salary first
    .limit(5)                              # top 5 only

result.show()

# SQL equivalent:
df.createOrReplaceTempView('staff')
spark.sql('''
    SELECT * FROM staff WHERE age > 30
    ORDER BY salary DESC LIMIT 5
''').show()
```

Scenario 1 Output

```
+-------+---+------+----------+
|   name|age|salary|department|
+-------+---+------+----------+
|Charlie| 45| 70000|        HR|
|  Grace| 41| 68000|        HR|
|   Jack| 38| 61000|        HR|
|  Helen| 35| 59000|     Sales|
|  Alice| 34| 55000|     Sales|
+-------+---+------+----------+
```

02 SCENARIO

Department Report

Objective: For each department, show average salary — only show depts with avg > 50000, sorted asc

Operations: filter + groupBy + having + sort

Scenario 2: Solution

```
from pyspark.sql import functions as F

result = df \
```

```
    .groupBy('department') \
    .agg(F.avg('salary').alias('avg_salary'),
         F.count('name').alias('employee_count')) \
    .filter(col('avg_salary') > 50000) \    # HAVING equivalent
    .orderBy(col('avg_salary').asc())

result.show()

# SQL with HAVING:
spark.sql('''
    SELECT department,
           AVG(salary) AS avg_salary,
           COUNT(name) AS employee_count
    FROM staff
    GROUP BY department
    HAVING AVG(salary) > 50000
    ORDER BY avg_salary ASC
''').show()
```

Scenario 2 Output

```
+----------+-----------+--------------+
|department|avg_salary |employee_count|
+----------+-----------+--------------+
|     Sales|    57000.0|             2|
|        HR|    66333.3|             3|
+----------+-----------+--------------+
(Finance avg = 47000 < 50000 -- excluded)
```

03
SCENARIO

IT Department Junior Staff

Objective: Find IT employees under age 30, show name & salary, sort by salary desc

Operations: filter + filter + select + sort

Scenario 3: Solution

```
result = df \
    .filter(col('department') == 'IT') \   # IT dept only
    .filter(col('age') < 30) \             # under 30 years old
    .select('name', 'age', 'salary') \     # only needed columns
    .orderBy(col('salary').desc())           # highest salary first

result.show()

# Single filter with &:
df.filter((col('department') == 'IT') & (col('age') < 30)) \
  .select('name', 'age', 'salary') \
  .orderBy(col('salary').desc())
  .show()
```

Scenario 3 Output

```
+-----+---+------+
| name|age|salary|
+-----+---+------+
|  Bob| 28| 42000|
|  Ivy| 26| 39000|
+-----+---+------+
(David age=23 is also IT and < 30)
```

Module 8: Master Reference & Interview Cheat Sheet

8.1 All DataFrame 04 Operations Summary

Operation	Type	Shuffle?	Key Syntax
filter() / where()	Narrow	No	df.filter((col('a')>3) & (col('b')=='x'))
isin()	Narrow	No	df.filter(col('dept').isin(['HR','IT']))
startsWith()	Narrow	No	df.filter(col('name').startsWith('A'))
endsWith()	Narrow	No	df.filter(col('name').endsWith('y'))
contains()	Narrow	No	df.filter(col('email').contains('@gmail'))
like()	Narrow	No	df.filter(col('name').like('A%'))
between()	Narrow	No	df.filter(col('age').between(25,40))
isNull()	Narrow	No	df.filter(col('salary').isNull())
isNotNull()	Narrow	No	df.filter(col('salary').isNotNull())
orderBy() / sort()	Wide	Yes	df.orderBy(col('salary').desc())
Multi-col sort	Wide	Yes	df.orderBy(col('a').asc(),col('b').desc())
sortWithinPartitions	Narrow	No	df.sortWithinPartitions('salary')
limit()	Narrow*	No	df.limit(10) — lazy transformation
withColumnRenamed()	Narrow	No	df.withColumnRenamed('old','new')
createOrReplaceTempView	Setup	-	df.createOrReplaceTempView('name')
spark.sql()	Varies	Varies	spark.sql('SELECT ... FROM name')

*** Note on limit()**

limit() appears narrow but Spark may need to collect some data to guarantee exactly N rows across partitions.

In practice: after orderBy().limit(N), Spark uses an optimized partial sort that avoids sorting all data.

limit() returns a DataFrame (lazy) — it is NOT an action. show(), write(), count() are actions.

Sateesh Says 💡

You have now mastered PySpark DataFrame 04 — filtering, sorting, chaining, Spark SQL, and lazy evaluation.

Practice Challenge 1: Use the assignment dataset, find all employees NOT in IT or Finance, age between 28 and 40, sorted by salary desc. Display only name, age, dept.

Practice Challenge 2: Register the dataset as a SQL view and solve the same challenge using pure Spark SQL.

Practice Challenge 3: Find the top 2 employees per department by salary using window functions (Window.partitionBy + row_number).

Lazy Evaluation Challenge: Write 5 transformations, call explain(True) before show(). Study the physical plan. Find where the filter was pushed down.

All the best! — Sateesh Pabbathi, MCT | www.levelupedu.net